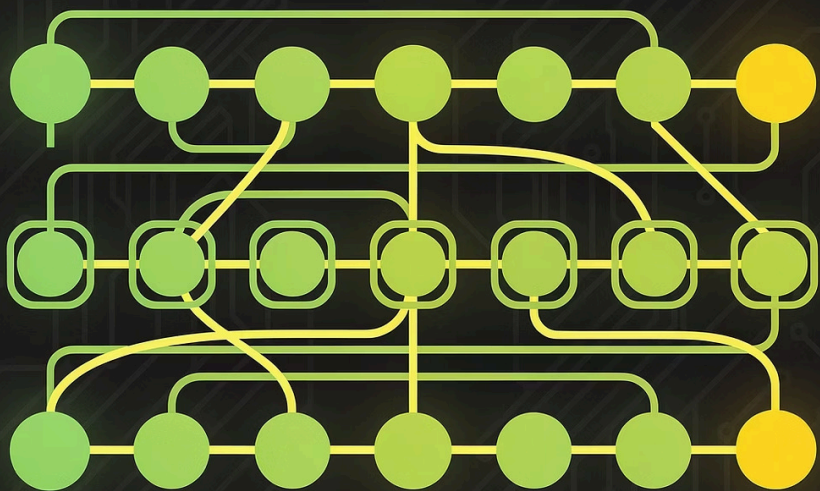




Projet IA BOT – Concevoir un agent, pas un simple chatbot

IA agentique · Web · MCP · Décision · Orchestration

Pourquoi ce projet existe

La distinction entre un chatbot classique et un agent intelligent est fondamentale pour comprendre l'architecture que vous allez concevoir. Cette différence ne repose pas sur la technologie utilisée, mais sur la capacité à prendre des décisions autonomes.

Chatbot classique

Répond à des questions prédéfinies

- FAQ structurée
- Patterns de reconnaissance
- Aucune autonomie
- Réponses scriptées

Agent intelligent

Prend des décisions contextuelles

- Analyse le contexte
- Évalue ses connaissances
- Choisit ses actions
- Raisonne et synthétise

📌 Un agent prend des décisions, il ne se contente pas de répondre.

Architecture globale du projet

L'architecture de votre système repose sur une orchestration intelligente de plusieurs composants. Chaque élément a un rôle précis dans le processus décisionnel de l'agent.

01

Frontend (Webapp)

Interface utilisateur et définition du contexte métier

02

Backend (Orchestrateur)

Logique de décision et coordination des flux

03

IA locale

Modèle open-source pour le raisonnement

04

Knowledge Base

Mémoire interne et connaissances statiques

05

Outils MCP

Actions externes ciblées (scraping)

Contraintes techniques du projet IA BOT (cadre imposé)

Ce projet impose un cadre technique précis pour garantir une architecture agentique cohérente. Voici ce qui est imposé, recommandé, et ce qui reste libre.

Frontend (Webapp)

Rôle

- Interface utilisateur
- Chat UI
- Exposition du contexte métier

Technologies attendues

- JavaScript / HTML / CSS
- Framework recommandé : React (ou équivalent JS)

Contraintes

- Interface web obligatoire
- Pas de simple CLI ou script

Le frontend ne contient pas la logique agentique.

Backend (Orchestrateur)

Rôle

- Orchestration du flux agentique
- Gestion de la Knowledge Base
- Décision tool / no tool
- Exposition des tools MCP

Technologies imposées

- Python obligatoire
- Framework recommandé : FastAPI ou Flask

Responsabilités clés

- Endpoint /chat
- Gestion des tools MCP (scraping)
- Communication avec l'IA locale

Toute la logique de décision se fait côté backend.

IA & Outils (Agent)

IA

- Modèle open-source local obligatoire
- Exemples : Ollama (LLaMA, Mistral, Qwen...)

Contraintes

-  Aucune API payante
-  Aucun service cloud externe

Outils (MCP)

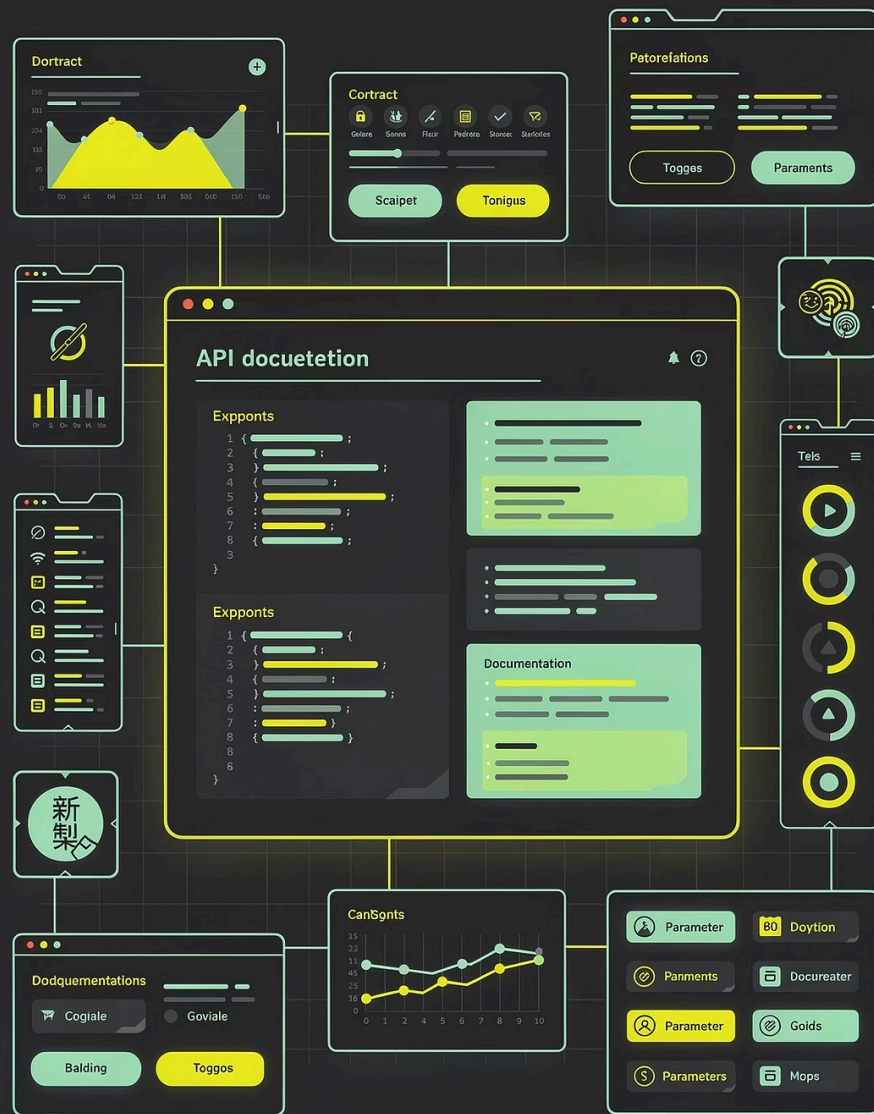
- Serveur Python simulant le Model Context Protocol
- Tools accessibles par l'agent : scraping, traitement de données, actions simples (mail, résumé...)

Le scraping simule l'appel à des outils externes.

Le flux agentique s'exécute intégralement côté backend Python

Messages clés :

- "Le backend Python est le cerveau de l'agent."
- "Le frontend est une interface, pas un moteur."
- "L'IA et les tools sont orchestrés, pas appelés directement par le front."



WEBAPP

Rôle réel de la webapp

La webapp n'est pas un simple conteneur d'interface. Elle définit le périmètre fonctionnel de votre agent et expose ses capacités de manière structurée. Comprendre ce qu'elle fait et ne fait pas est essentiel pour concevoir une architecture cohérente.

✓ Ce que la webapp FAIT

- Définit le **contexte métier** de l'application
- Expose les **possibilités** de l'agent
- Contient la **connaissance statique** structurée
- Gère l'interface et l'expérience utilisateur
- Établit le contrat d'usage avec l'agent

Ce qu'elle NE FAIT PAS

- Stocker des **données temps réel** (vols, météo)
- Exécuter des actions externes directement
- Remplacer la base de connaissances
- Gérer la logique décisionnelle
- Scraper systématiquement des sources

Knowledge Base : la mémoire interne

La Knowledge Base (KB) est le socle de votre agent. Elle contient tout ce que l'agent sait **sans avoir besoin d'agir**. C'est la première source consultée avant toute décision d'action externe.

Définition

Ensemble structuré de connaissances statiques, contextuelles et métier que l'agent peut consulter instantanément pour répondre ou prendre des décisions.

Exemple de structure kb.json

```
{
  "destinations": {
    "Lisbonne": {
      "meilleures_periodes": ["mars", "avril", "mai"],
      "temperature_moyenne_mars": "16-20°C",
      "attractions": ["Belém", "Alfama", "Bairro Alto"]
    }
  }
}
```

📌 Sans Knowledge Base, il n'y a pas de décision intelligente possible.



Pourquoi le scraping existe (et ses limites)

Le scraping n'est pas l'objectif du projet, mais un **outil parmi d'autres**. Dans un contexte pédagogique sans API payantes, il sert de substitut pour obtenir des données dynamiques. Son utilisation doit toujours être justifiée par un besoin réel.

1

Principe

Le scraping intervient **uniquement** lorsque l'information n'existe pas dans la Knowledge Base interne.

2

Contrainte pédagogique

Absence d'API payantes → scraping comme substitut éducatif pour données temps réel.

3

Règle d'or

Ne **jamais** scraper de manière systématique. Toujours vérifier la KB d'abord.



Le flux agentique CORRECT

Voici le processus décisionnel complet que votre agent doit suivre. Chaque étape est critique et doit être implémentée avec rigueur. L'ordre est fondamental : on ne va jamais chercher une information externe avant d'avoir vérifié ce qu'on sait déjà.



1. Requête utilisateur

Réception et analyse de la demande en langage naturel



2. Classification de l'intention : intention sociale ou intention métier

Analyse et catégorisation de l'intention utilisateur. Si intention sociale, réponse directe du LLM. Sinon, étape 3.



3. Recherche dans la KB

Consultation prioritaire de la base de connaissances interne



4. Décision

Évaluation : l'information disponible est-elle suffisante ?



5. Appel tool MCP (si nécessaire)

Exécution ciblée d'une action externe justifiée



6. Fusion & raisonnement

Intégration des données et analyse contextuelle



7. Réponse finale

Formulation d'une réponse raisonnée et contextualisée



CAS PRATIQUE

Exemple concret : Thématique Voyage

Prenons un cas d'usage réaliste pour illustrer le processus décisionnel de l'agent. Ce scénario va nous permettre de comprendre comment l'agent navigue entre ses connaissances internes et les informations à rechercher.

Requête utilisateur

"Je veux partir à Lisbonne fin mars depuis Paris"



Trouvé dans la KB

- Période idéale pour Lisbonne (mars-mai)
- Température moyenne en mars
- Attractions principales
- Conseils pratiques de voyage



Manquant (données dynamiques)

- Prix des vols Paris-Lisbonne
- Disponibilités fin mars
- Compagnies aériennes opérant
- Horaires de vol spécifiques

DÉCISION

Décision de l'agent (zoom)

L'agent ne se contente pas d'exécuter des actions par défaut. Il **analyse** ce qu'il sait, **identifie** ce qui lui manque, et **décide** de manière rationnelle quelles actions externes sont nécessaires. Cette capacité décisionnelle est ce qui différencie un agent d'un simple script.

Information recherchée	Disponible en KB ?	Action de l'agent
Période idéale pour visiter Lisbonne	✓ Oui	Utilisation directe de la KB, pas de scraping
Température moyenne en mars	✓ Oui	Utilisation directe de la KB, pas de scraping
Attractions touristiques principales	✓ Oui	Utilisation directe de la KB, pas de scraping
Prix des vols Paris-Lisbonne fin mars	✗ Non	Scraping ciblé via FlightScraper
Disponibilités de vol	✗ Non	Scraping ciblé via FlightScraper

📌 L'agent choisit, il n'exécute pas par défaut. Chaque action externe doit être justifiée par un besoin informationnel réel.

Scraping ciblé via MCP

Le Model Context Protocol (MCP) structure l'interaction entre l'agent et ses outils externes. Chaque tool a un contrat clair : des inputs structurés et des outputs normalisés. Le scraping devient ainsi une opération contrôlée et prévisible.

Input structuré

```
{
  "tool": "FlightScraper",
  "params": {
    "origin": "Paris",
    "destination": "Lisbonne",
    "date_start": "2024-03-20",
    "date_end": "2024-03-31"
  }
}
```

Output structuré

```
{
  "flights": [
    {
      "price": "89€",
      "company": "EasyJet",
      "date": "2024-03-25",
      "duration": "2h50"
    }
  ],
  "scraped_at": "2024-01-15T10:30:00Z"
}
```

Le tool retourne uniquement les **données essentielles**, pas le HTML brut. L'agent peut alors intégrer ces informations dans son raisonnement.

API Tool



Fusion & raisonnement

L'intelligence de l'agent ne réside pas dans sa capacité à collecter des données, mais dans sa capacité à les **interpréter**, les **contextualiser** et les **synthétiser**. Les données brutes deviennent des insights actionnables.



Données KB

Mars = période idéale
Température 16-20°C
Attractions : Belém, Alfama



Données scrapées

Vol EasyJet : 89€
Date : 25 mars
Durée : 2h50



Raisonnement IA

Analyse des conditions
Mise en contexte
Évaluation globale

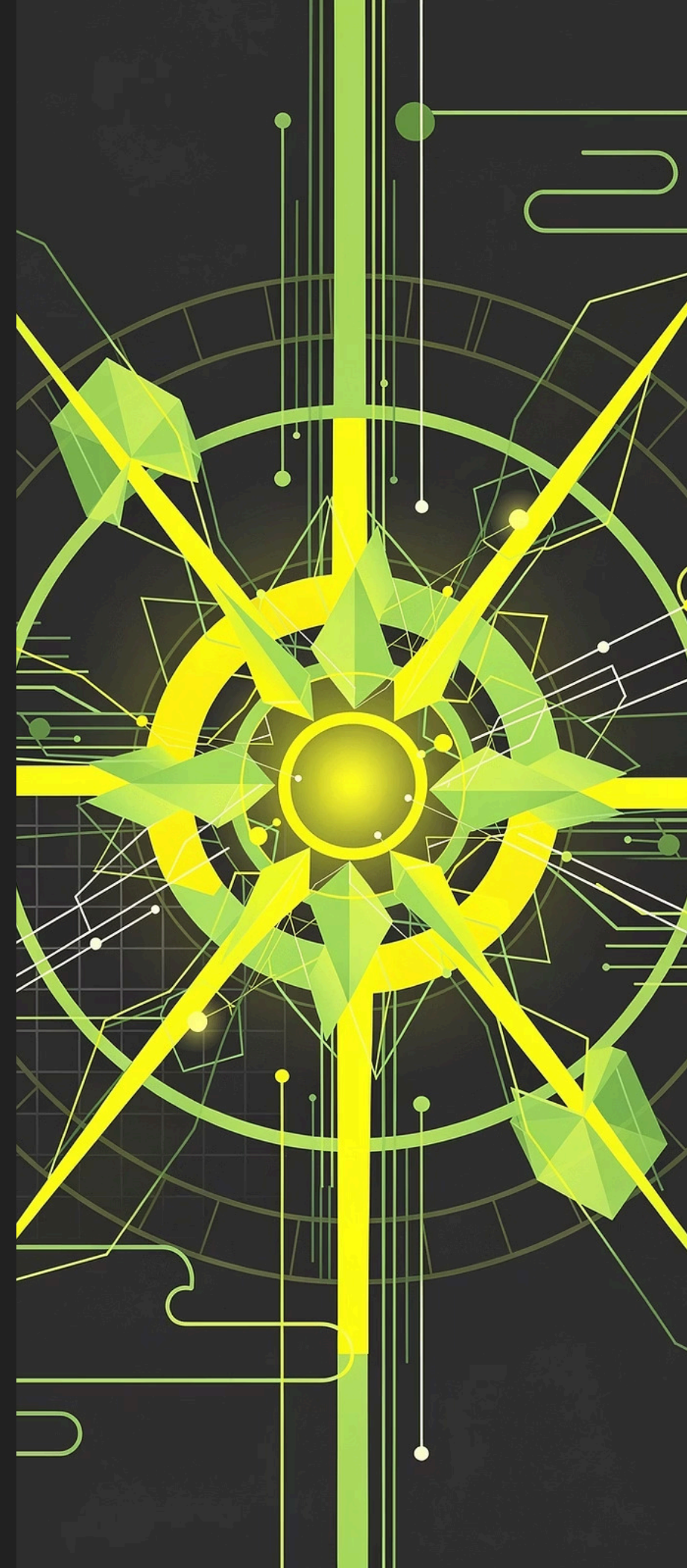


Réponse synthétique

Recommandation
contextualisée et
argumentée

Exemple de réponse de l'agent :

"Mars est une période idéale pour Lisbonne avec des températures agréables de 16-20°C. J'ai trouvé un vol EasyJet à 89€ le 25 mars (2h50 de vol). Je vous recommande de visiter Belém pour ses monuments historiques et Alfama pour son ambiance authentique."



Bonne vs mauvaise implémentation

La différence entre un projet réussi et un projet raté réside dans la rigueur architecturale et la justification de chaque choix technique. Voici les critères qui distinguent une implémentation professionnelle d'une implémentation approximative.

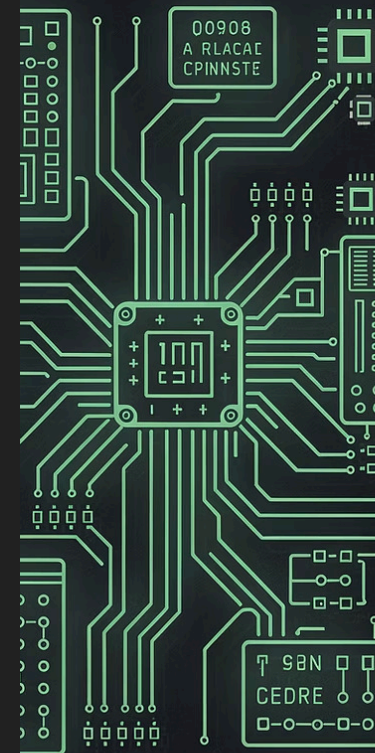
Mauvaise implémentation

- **Scraping massif et systématique** à chaque requête
- Aucune consultation de la KB
- Données brutes envoyées au LLM sans traitement
- Pas de logique décisionnelle
- Actions externes non justifiées
- Performance médiocre (latence élevée)
- Code spaghetti sans séparation des responsabilités

✓ Bonne implémentation

- **Scraping ciblé** uniquement si nécessaire
- KB consultée en priorité systématiquement
- Données structurées et filtrées avant raisonnement
- Logique décisionnelle explicite et traçable
- Chaque action externe est justifiée
- Performance optimisée (cache, lazy loading)
- Architecture claire avec séparation des couches

CLEAN CODE



MESSY CODE



Ce qui sera valorisé dans l'évaluation

Votre projet sera évalué sur sa capacité à démontrer une compréhension profonde des concepts d'IA agentique et d'architecture logicielle. La technique au service de l'intelligence, pas l'inverse.



Architecture claire

Séparation nette des responsabilités entre webapp, KB, orchestrateur et tools MCP. Diagrammes d'architecture explicites et code structuré.



Logique de décision

Processus décisionnel explicite et traçable. L'agent doit pouvoir expliquer pourquoi il consulte la KB ou appelle un tool externe.



Tools du MCP

Chaque outil MCP doit avoir une raison d'exister.



Clarté de la démo

Démonstration live montrant le flux complet avec des exemples contrastés (KB suffisante vs nécessité de scraping).



Documentation technique

README exhaustif, documentation du code, schémas d'architecture, et explication des choix techniques.



Tests et robustesse

Gestion des erreurs, cas limites, et validation des données. L'agent doit gérer élégamment l'échec d'un tool.

Erreurs classiques à éviter

Voici les erreurs récurrentes observées dans les projets d'étudiants. Les identifier maintenant vous évitera des réécritures complètes plus tard. Chacune de ces erreurs trahit une incompréhension fondamentale de l'architecture agentique.

Scraper "pour scraper"

Utiliser le scraping comme objectif principal plutôt que comme moyen.
Le scraping n'est **jamais** une fin en soi, seulement un outil contextuel.

Tout envoyer au LLM

Bombarder le modèle avec des données brutes non filtrées en espérant qu'il trie lui-même. Le traitement doit être fait **avant** le LLM.

Pas de séparation statique/dynamique

Confondre connaissances statiques (KB) et données temps réel (tools). Cette distinction est architecturalement fondamentale.

Chatbot déguisé en agent

Créer un système de questions-réponses scriptées en le présentant comme un agent. Un agent **décide**, il ne suit pas un script.

Ignorer la Knowledge Base

Ne pas consulter la KB ou la sous-utiliser. C'est le socle de l'intelligence de votre agent, pas une option.

Pas de gestion d'erreur

Ne pas prévoir les échecs de scraping ou les timeouts. Un agent robuste gère l'incertitude avec élégance.

Votre agent n'est pas un aspirateur à données

C'est un décideur.

Vous allez construire un système qui **réfléchit avant d'agir**, qui **évalue ses connaissances avant de chercher**, et qui **raisonne au lieu de compiler**. C'est cette exigence architecturale qui transformera votre projet d'un exercice technique en une démonstration d'intelligence artificielle agentique.

L'excellence ne réside pas dans la quantité de données traitées, mais dans la **pertinence des décisions prises**. Un agent qui sait quand **ne pas agir** est aussi intelligent qu'un agent qui sait agir.

📌 **Prochaines étapes** : Architecture détaillée · Implémentation de la KB · Développement des tools MCP · Tests et validation